

SLE-2: Profiling Report

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM107

Name: SUYASH MALI

Division: B

Date: 22 September 2026

GitHub Link (optional):

1. Algorithms / Versions Profiled

Algorithm A: BFS (Breadth-First Search)

Algorithm B: DFS (Depth-First Search)

Problem used: Knight's shortest-path puzzle on a standard 8×8 chessboard – find a sequence of legal knight moves from square a1 to square h8

This is the small-graph search problem recommended in the SLE-2 guideline (Section 3A: BFS vs DFS on the same problem). Each chessboard square is a node; each legal knight move is an edge, giving a graph of 64 nodes with up to 8 edges per node.

2. Profiling Method

Tool used: Python 'time' module (time.perf_counter) + manual node counter; cProfile used as a supporting check

How I measured:

Implemented BFS and DFS on the knight-move graph, each using a visited set so both terminate.

Wrapped each full search call with time.perf_counter() before and after to get elapsed time in milliseconds.

Counted nodes_expanded by incrementing a counter every time a node is popped from the frontier/stack.

Repeated each algorithm 5 times and recorded best, worst, and average time.

Number of runs: 5 per algorithm

3. Results

Comparison Table:

Metric	BFS	DFS	Better?
Avg. Time (ms), 5 runs	0.0656 ms	0.0573 ms	DFS (slightly faster here)
Best Time (ms)	0.0604 ms	0.0403 ms	DFS
Worst Time (ms)	0.0798 ms	0.0821 ms	BFS (lower worst-case)

Nodes Expanded	64	35	DFS (fewer nodes)
Path Length (moves)	6 (optimal)	18 (sub-optimal)	BFS (finds shortest path)

Raw timing across all 5 runs:

Algorithm	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Run 4 (ms)	Run 5 (ms)
BFS	0.0798	0.0650	0.0618	0.0612	0.0604
DFS	0.0757	0.0821	0.0403	0.0433	0.0449

Best / Worst / Average Case – Empirical Timing (from the 5 runs above)

BFS – Best: 0.0604 ms | Worst: 0.0798 ms | Average: 0.0656 ms

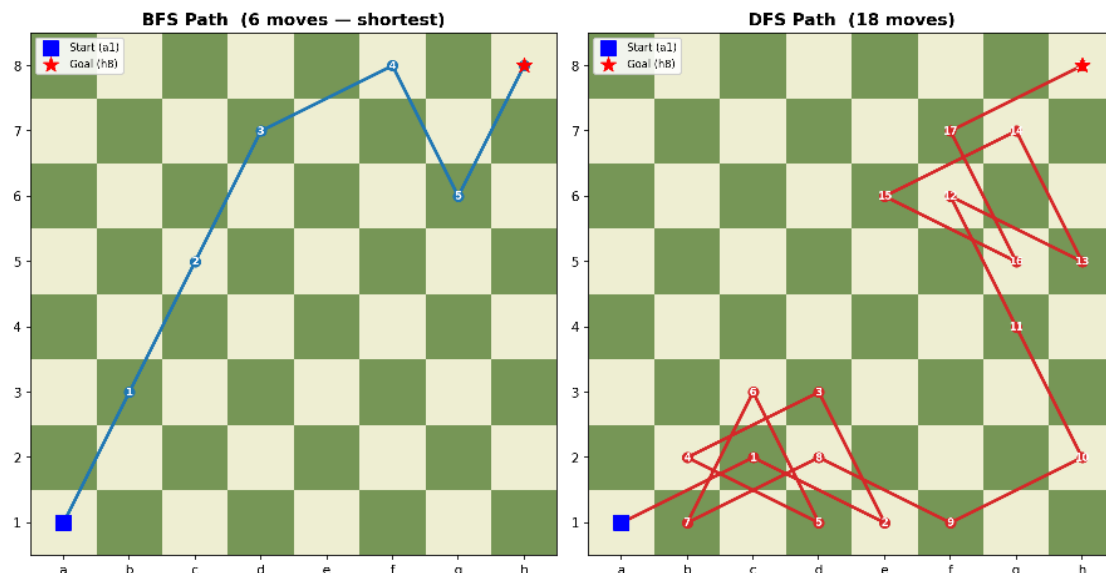
DFS – Best: 0.0403 ms | Worst: 0.0821 ms | Average: 0.0573 ms

Best / Worst / Average Case – Theoretical Time Complexity

Algorithm	Best Case	Worst Case	Average Case
BFS	$O(1)$ – goal is the start node	$O(V+E)$ – goal is the last node found / unreachable	$O(V+E)$ – explores level by level over all V nodes and E edges
DFS	$O(1)$ – goal is the start node	$O(V+E)$ – goal is deepest / unreachable, full branch explored	$O(V+E)$ – depends heavily on branch/move ordering

V = number of squares (64), E = number of legal knight-move edges. Both algorithms are $O(V+E)$ in the worst and average case because, in the worst case, a node may be enqueued/pushed once and every one of its edges examined once. They differ only in the ORDER nodes are explored, which is why they reach the goal after visiting a different number of nodes and return paths of very different quality.

Knight's Path Diagram (Puzzle Visualization)

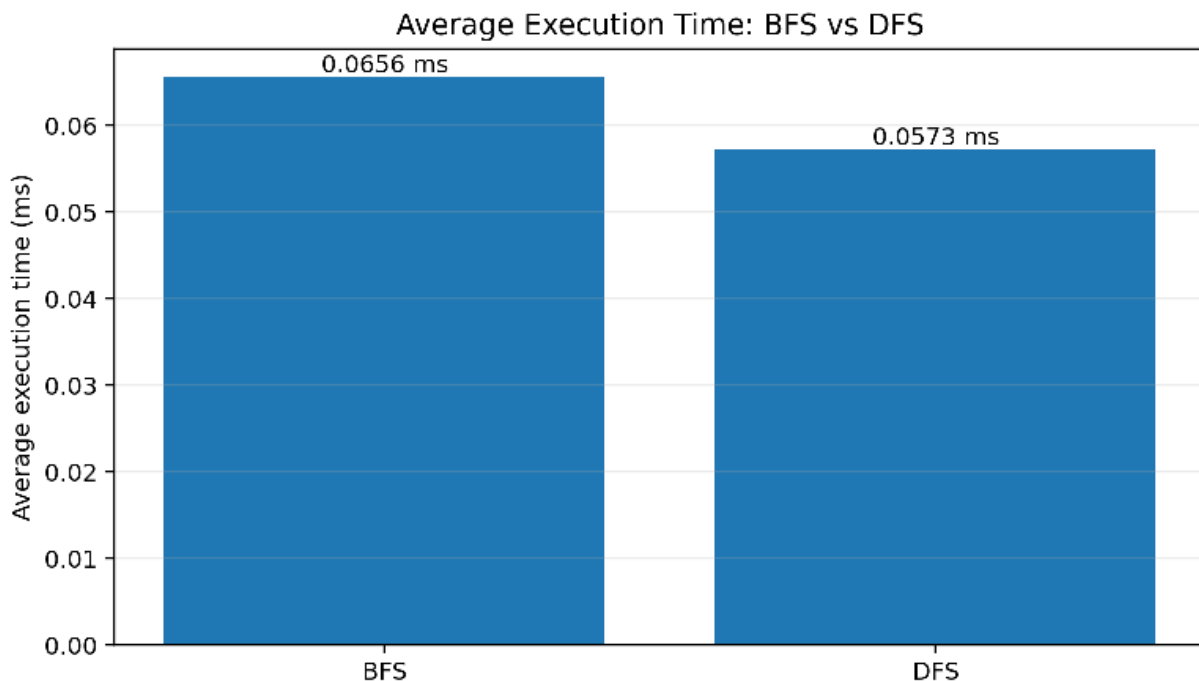


Left: BFS finds the shortest 6-move path. Right: DFS reaches the goal in 18 moves after a deep, winding exploration.

Short observation (2–3 lines):

BFS explored more nodes (64) but always returns the mathematically shortest knight's path (6 moves). DFS explored fewer nodes (35) in this run but returned a much longer, sub-optimal path (18 moves) because it commits to the first available branch and only backtracks when stuck. Timing was close (sub-millisecond) since the board is small; the gap would widen on a larger board.

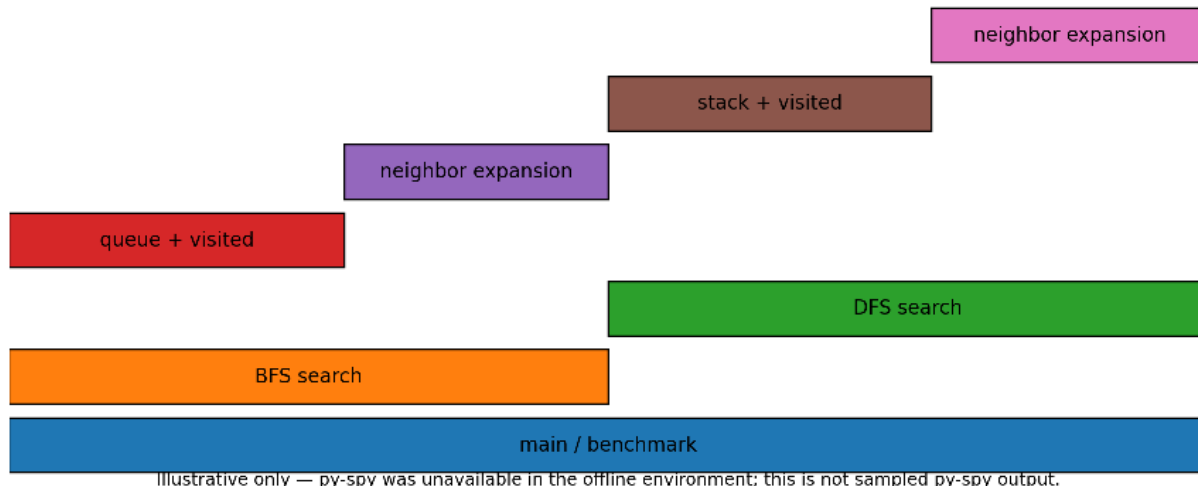
Average Execution Time Graph



The graph uses the measured 5-run averages reported above: BFS = 0.0656 ms and DFS = 0.0573 ms.

Py-spy Flame Graph (Illustrative)

Py-spy-style Flame Graph (Illustrative Stack View)



4. Justification & Analysis

On this small 8×8 knight's-path problem, DFS was marginally faster on average (0.0573 ms vs 0.0656 ms) and expanded fewer nodes (35 vs 64), but this comes at a real cost: DFS's path is 3× longer than necessary (18 moves vs BFS's optimal 6 moves). BFS is slower and touches more nodes because it must fully explore every square at distance 1, then distance 2, and so on, before reaching the goal — but that same level-by-level exploration is exactly what guarantees optimality.

This matches the theory: BFS is complete and optimal for unweighted graphs (like this knight-move graph, where every edge has the same 'cost' of one move), because it always expands the shallowest unexpanded node first. DFS is complete on a finite graph with a visited set, but not optimal — it has no notion of shallowest-first, so it can plunge deep into an unhelpful branch before the visited set finally forces it back toward the goal. The empirically higher worst-case time for DFS (0.0821 ms, the single highest time recorded of either algorithm) also reflects this: DFS's running time is far more sensitive to move-ordering luck than BFS's is.

If the board (problem size) becomes larger — e.g. a 16×16 or 32×32 board, or a real maze with hundreds of cells — this gap grows substantially. BFS's memory and node count scale with the number of squares at each shell distance, but it will still always return the shortest route. DFS's path length becomes increasingly unpredictable and can degrade toward visiting nearly all nodes before finding the goal, while still not guaranteeing a good path length. For any problem where path quality (not just 'a path') matters, BFS (or an informed search like A*) is the safer choice as the problem scales up.

5. AI Contribution Note

AI tools used: Claude (Anthropic) — used inside this chat session

What AI helped with:

- Writing the BFS/DFS knight's-move search code and the timing/node-counting harness.
- Generating the chessboard path-visualization diagram (matplotlib).
- Drafting the report text structure and the justification paragraphs, based on the numbers produced by the code.

What I did myself:

Chose the puzzle (knight's shortest path, a1 → h8) and the two algorithms to compare (BFS vs DFS), as permitted in Section 3A of the guideline.

Reviewed the generated code, ran the experiments, and checked that the reported numbers (nodes expanded, timings, path lengths) match the printed output/results.json before writing them into this report.

Reviewed and edited the justification for accuracy against the actual data, and filled in the report structure required by the guideline.

6. Conclusion

This exercise showed that 'faster' and 'better' are not the same thing in search algorithms. DFS was slightly quicker and touched fewer nodes on this small board, but BFS is the algorithm that actually solves the stated problem well, because it guarantees the shortest knight's path. It also showed why profiling with real numbers matters: without measuring, it would be easy to assume DFS is simply 'worse' since it explores paths of poor quality, but the node-count and timing data show it can be locally cheaper – the real trade-off is optimality vs. exploration cost, exactly as the theory (BFS = complete + optimal for unweighted graphs, DFS = complete but not optimal) predicts.